

Sittie-Typpie Reader / Writer

CSCI 439 - Project Report

by Patrick McGrath

April 4, 2025

Overview

Sittie-Typpie is a turn-based text file-based chat log script. Its purpose is to simulate the inner structure of a potential user-friendly chat system. That system would be the interface for users to do various things like leave messages. This program facilitates what information that system would have access to based on previous interactions and also how to handle and record current interactions to the same log.

The name is because, like Walkie-Talkies, Sittie-Typpie uses a shared medium between all users who must take turns to use it. However Walkie-Talkies are used by talking and can be used during walking. Whereas Sittie-Typpie must be used by typing and while sitting.

Language Specification

Literals and Tokens

Literals:

USERID:	U([0-9][0-9][0-9])
ENTRYID:	E([0-9][0-9][0-9])
TEXTLIT:	["]^[^"]*["]

Tokens:

OPEN:	{{
CLOSE:	}}
INVITE:	INV
LOGIN:	LGN
MESSAGE:	MSG
EVENT:	EVT

RESPONSE:	RSP
THUMBSUP:	TMUP
THUMBSDOWN:	TMDN
SMILEYFACE:	SMILEY
BLANK:	BLNK

*Note: The internal portion of TEXTLIT is “neither double-quote nor space”.

Grammar

```
start:
    sequences

sequences:
    sequence sequences

sequence:
    OPEN LOGIN USERID entries CLOSE ||
    OPEN LOGIN USERID CLOSE

entries:
    entry entries

entry:
    messageentry || evententry || responseentry || inviteentry

messageentry:
    OPEN ENTRYID MESSAGE TEXTLIT CLOSE

evententry:
    OPEN ENTRYID EVENT TEXTLIT TEXTLIT TEXTLIT CLOSE ||
    OPEN ENTRYID EVENT TEXTLIT BLANK TEXTLIT CLOSE ||
    OPEN ENTRYID EVENT TEXTLIT TEXTLIT BLANK CLOSE ||
    OPEN ENTRYID EVENT TEXTLIT BLANK BLANK CLOSE

responseentry:
    OPEN ENTRYID RESPONSE ENTRYID TEXTLIT CLOSE ||
    OPEN ENTRYID RESPONSE ENTRYID reactiontoken CLOSE

inviteentry:
    OPEN ENTRYID INVITE USERID CLOSE
```

```
reactiontoken:
    THUMBSUP ||
    THUMBSDOWN ||
    SMILEYFACE
```

Notes

Aside from the token and grammar rules above, the structure of a *.sittie* file must also satisfy the following requirements.

All TEXTLITs are recorded with the spaces substituted with “_SPACE_”. This is undone on read, and redone on write. Therefore, although all characters in “_SPACE_” are allowed, they are forbidden together.

All users, except the first one, must have been declared in an inviteentry rule, before that user can log in. Users can be invited any number of times beyond the first.

In every entry rule, the first ENTRYID token is that entry’s identifier. It’s integer portion must be exactly 1 greater than the previous ENTRYID, starting at 000.

Each evententry rule has 3 fields. The first is the name of the event and must be a TEXTLIT. The 2nd field is the location and is optional. If chosen, it is also a TEXTLIT. Otherwise it is a BLANK token. The 3rd field is time and it has the same restrictions as the 2nd field.

Each responseentry has a 2nd ENTRYID token field that identifies the entry that is being responded to. This 2nd ENTRYID must be present previously in the script. Each responseentry has the option of either a TEXTLIT or several reactiontokens.

Structure

Each script is a series of sequences. Sequences consist of a user logging in, followed by zero or more entries. Sequences serve the purpose of identifying who made each entry, kind of like a scope. Each sequence must start with an opener and end with a closer.

Entries are the heart of the script. They record all the interactions by users. Some entry types make references to either previous entries (responses) or users previous or not (invitations). Both of these values do not care which sequence these values were previously declared in. Each entry must also be opened and closed.

The Program

The program is written in C++. Command line is used to interact with the user, while logs are read through user-specified files. All interactions are via command line in linux.

How to use it

To compile, in the source code directory, enter:
make

To uninstall and remove compiled objects, enter:
make clean

Compilation will produce two versions of the program; the regular one and a test version with an excess of extra debugging information:

projx and projxTest

To run the program, make sure you are the only person currently using the chat log you intend to open. Then, enter:

./projx or ./projxTest

Steps:

- 1:
 - The program asks for the filename to open or create.
 - If the file does not exist, it is created, otherwise it is opened.
 - If it is opened, it is tokenized and parsed. If not successful, the program exits.
2.
 - The program asks for a userID number, front-padded with 0's.
 - If the user is not in the log, the program exits.
 - If the user is in log, relevant log information is printed.
 - The program also printed the beginning of this sequence to log.
3.
 - While the user has not selected to quit the program, the program will ask for a command and execute it
4.
 - The commands are:

h	Prints these commands (no interaction)
l	Prints the in-memory chat log information (no interaction)
m	To record a message entry.
e	To record an event entry
r	To record a response entry
i	To record a user invitation

q To quit the program. (no interaction)

Command Interactions:

- M: The program will ask for text to enter as message.
- E: The program will ask for the name of the event.
 The program will ask for the location of the event, optional.
 The program will ask for the time of the event, optional.
- R: The program will ask for the entryID of the entry being responded to.
 The program will ask for either text or to select an optional response.
- I: The program will ask for a userID to invite.

How it works

The program identifies tokens word-by-word from the specified log file. This token array is parsed using recursive descent. Recursive descent parses the log by checking for patterns of grammar rules in the order they appear in the grammar. That is, the log consists of a list of sequences each with an internal list of entries. The program will, while there are still tokens, try to parse a sequence. Within the sequence parsing, while the end of the sequence has not yet been found, it will try to parse entries.

Logs are stored in "logs/" and all have the extension ".sittie". When the program asks for a filename, it will look in this folder, and it expects the extension to not be specified by the user.

The parse produces an entries array and a users array. When the user selects an option that needs info from these, they are searched. For example, a response needs to have an entryID in its first field that is already in the entries array.

When a user selects a command that does printing to the log, the needed info is got from the arrays and the user. Near the beginning of the program, after the file is parsed and the userID is validated, the opening of the sequence is printed. Then, the program handles all the users commands, printing to file as needed. At the end of commands, the program finally prints the closing token for the sequence. Each sequence corresponds to a user's session on this program.

Because the program simulates what another system would have access to, a shortcut is taken with recording TEXTLITS. It is slightly easier to tokenize and parse single word tokens. So, the program automatically swaps out spaces with an unlikely string "_SPACE".

If the user inputs incorrect commands, or an improperly formatted log file is used, the program will have unintended results that could include printing incomplete content to the log file.

Conclusion

Designing the script and program was interesting and challenging. Recursive descent is a predictable way of extracting information from formatted data. By knowing precisely how data may be formatted, a robust in-memory analysis can be done.

An interesting observation is that sequences make a scope for an entry's creator. Whereas entries are all global, regardless of creator scope. I wonder if there is any programming example of a generally scoped language that lets you declare globals at will, like in an if statement.

Problems / Improvements

The first and obvious improvement would be to build a graphic interface with more options than are available here.

The program could also have a system of logging in with a password for security. It could encrypt messages and have a simple database for multiple messages. Perhaps an inbox, where when something like an invite happens, the program also logs a related entry in that user's inbox.

Sample script

This is an example of 3 users talking.

```
{{ LGN U000
{{ E000 MSG
"This_SPACE_is_SPACE_an_SPACE_example_SPACE_of_SPACE_how_SPACE_a_SPACE_chat_SPACE_
between_SPACE_3_SPACE_users_SPACE_may_SPACE_look."
}}
{{ E001 MSG
"now_SPACE_user_SPACE_U000_SPACE_will_SPACE_invite_SPACE_user_SPACE_U001"
}}
{{ E002 INV U001
}}
}}
```

```
{{ LGN U001
{{ E003 MSG
"now_SPACE_U001_SPACE_has_SPACE_logged_SPACE_in._SPACE_they_SPACE_will_SPACE_respo
nd_SPACE_to_SPACE_the_SPACE_invite_SPACE_with_SPACE_a_SPACE_thumbsup"
}}
{{ E004 RSP E002 TMUP
}}
{{ E005 MSG
"this_SPACE_user_SPACE_will_SPACE_now_SPACE_invite_SPACE_a_SPACE_3rd_SPACE_user_SP
ACE_with_SPACE_a_SPACE_disjoint_SPACE_number"
}}
{{ E006 INV U111
}}
}}
{{ LGN U111
{{ E007 MSG
"now_SPACE_the_SPACE_3rd_SPACE_user_SPACE_is_SPACE_here._SPACE_they_SPACE_will_SPA
CE_log_SPACE_out_SPACE_and_SPACE_log_SPACE_in_SPACE_without_SPACE_doing_SPACE_anyt
hing"
}}
}}
{{ LGN U111
}}
{{ LGN U111
}}
{{ LGN U000
{{ E008 MSG "_SPACE_first_SPACE_user_SPACE_again._SPACE_end"
}}
}}
```